

Fixing the buy flow

The purchase path has two problems that compound each other: the transaction handler reports success it hasn't verified, and the gate in front of it is copy-pasted across five files that have already drifted apart. Taken together they produce wrong money messaging and a measurable hole in analytics.

08 The transaction handler

One file, `src/component/modals/BuyNowModal.tsx`. Here is what actually happens when a user clicks Buy Domain, with the problem steps marked.

1 Auth, provider, chain map, signer

Token check, MetaMask check, provider/ABI lookup per chain, signer address. All fine.

2 Wrong-network check — bails instead of switching

Sets `isWrongNetwork`, stops, and relabels the button to "Switch Network". The user must click a second time to switch, then a third to buy.

3 Balance check compares the wrong number

Compares balance against `priceInWei` (derived from `pricePerToken`) but sends `safeEthAmount` (from the contract's own `getRequiredNativeAmount` plus 2%). Different quantities. Ignores gas entirely, so the check passes and the transaction still reverts.

4 Hardcoded 2% slippage buffer

`requiredNativeAmount.mul(102).div(100)`. A magic number, not configurable, never disclosed to the user, and the overage is not refunded by the contract path.

5 Send the transaction

`contract.buy(listingId, userAddr, nativeSentinel, {value})`. Returns as soon as it is broadcast — not mined.

6 Tell the backend before the chain confirms

POSTs the purchase with `tx.hash` while the transaction is still pending. And the catch block toasts an error but **does not return** — so execution falls straight through to the success toast below.

7 Declare success, then close the modal

"Purchase initiated" fires unconditionally — including after step 6 has already failed — and `handleClose()` dismisses the modal.

8 Await confirmation against a dismissed UI

Only now does `tx.wait(1)` run. Its "Transaction confirmed" and "Transaction failed after sending" toasts land on a modal the user closed several seconds ago. `receipt.status` is never checked, so a mined-but-reverted transaction is reported as confirmed.

CORRECTNESS A reverted purchase is reported as a successful one

Two independent causes. First, `tx.wait(1)` resolves for any mined transaction, and the code never inspects `receipt.status` — so an on-chain revert takes the success branch. The delist flow in `domain-self-item` *does* check `receipt.status === 1`, so the correct pattern already exists in the repo; the buy path just doesn't use it.

Second, the backend is told about the purchase at step 6 regardless of outcome. The reconciler eventually corrects the record — it verifies the `NewSale` event properly, and that part is well built — but only after up to 7 retries at one-minute intervals. In between, the user has been told they bought a domain they may not own.

`BuyNowModal.tsx:197-218` · cf. `domain-self-item/index.tsx:286`

CORRECTNESS Double-submit is possible

The Buy button carries no `disabled` attribute. `isTransactionLoading` only renders a `<Loader />` alongside it. Whether a second click lands depends entirely on whether that overlay happens to cover the button — a CSS accident, not a guard. A double-click during the wallet round-trip can open two wallet prompts for the same listing.

`BuyNowModal.tsx:283-291`

UX Every failure reads the same

The outer catch emits "Something went wrong. Please try again." for everything — a user rejecting in MetaMask (code 4001), an on-chain revert, an RPC timeout, and insufficient funds all produce identical copy. A rejection is not an error and shouldn't be toasted as one; the other three each need a different next action from the user.

There is also no link to the transaction on a block explorer at any point, so a user whose purchase is mid-flight has no way to check it themselves.

`BuyNowModal.tsx:220-225`

What the flow should be

The ordering principle is simple: **nothing is claimed until the chain confirms it**, and the modal stays open and owns the state until it resolves.

```

// 1 – guard the entry, not just the render
if (isSubmitting) return;
setPhase('idle' 'confirming'); // button is disabled from any phase != idle

try {
  // 2 – switch the chain inline; don't make the user click twice
  if (network.chainId !== expected) {
    await switchChain(expected); // then continue, no re-click
  }

  // 3 – quote and check against the same number you will send
  const required = await contract.getRequiredNativeAmount(priceInUSD);
  const value = withBuffer(required); // buffer from config, shown in UI
  const gas = await contract.estimateGas.buy(...args, { value });
  if (balance.lt(value.add(gas.mul(gasPrice)))) // include gas
    return fail('insufficient-funds', { needed: value, have: balance });

  // 4 – send
  setPhase('pending');
  const tx = await contract.buy(...args, { value });
  showTxLink(tx.hash); // explorer link, immediately

  // 5 – CONFIRM FIRST
  const receipt = await tx.wait(1);
  if (receipt.status !== 1) return fail('reverted', { hash: tx.hash });

  // 6 – only now is it real: persist, and let this one throw
  await setBuyDomain({ ...payload, blockchainTxHash: tx.hash }).unwrap();

  // 7 – and only now say so
  setPhase('success');
} catch (e) {
  if (e.code === 4001) return setPhase('idle'); // user cancelled – not an error
  fail(classify(e), { hash: tx?.hash });
}

```

One decision this forces. If the wallet confirms but the backend POST at step 6 fails, the user owns the domain and your database doesn't know. Today that's masked because the POST happens first and its failure is swallowed. Doing it properly means the modal has to say "purchase confirmed on-chain, syncing your account" and retry in the background — which is honest, and matches the "10 to 15 minutes" language already in the order emails.

The reconciler is the safety net that makes this safe to do: it independently verifies the `NewSale` event and will correct the record within a minute or two even if the POST never lands. Lean on it rather than working around it.

Worth being explicit that **the backend already implements this pattern properly** — `buyDomain` takes an atomic conditional reservation (`Available → In Progress` , throwing `ConflictException` if a concurrent buyer wins), writes the row at `blockchainStatus: 'Pending'` , and lets the cron confirm it. So none of this is a redesign. The server half of the standard intent-then-reconcile flow is built and correct; the client half is skipping steps in it.

21 The duplicated buy gate

Before the handler above ever runs, a preconditions ladder decides whether to open the wallet modal or the buy modal. That ladder is copy-pasted, verbatim, into five files.

SURFACE	FILE	GATE	TRACKS ANALYTICS
Grid card	<code>domain-item/page.tsx</code>	copy 1	yes — 6 calls
Homepage carousel	<code>slider/slider.tsx</code>	copy 2	none
Watchlist	<code>watchlist/page.tsx</code>	copy 3	none
Homepage inline	<code>pages/index.tsx</code>	copy 4	none
Seller portfolio	<code>seller-portfolio/[slug].tsx</code>	copy 5	none

DATA LOSS Four of five buy surfaces record nothing

This is the concrete cost of the duplication, and it is worse than a maintenance smell. Only the grid card fires `setAnalytics` — clicks and modal-opens on click, saves and unsaves on the heart, and a view on 50% viewport intersection. The other four copies were pasted before or without that block and never gained it.

So a click from the homepage carousel, the watchlist, or a seller's portfolio is **invisible**. Those numbers feed the seller analytics dashboard and, through `clicks•0.6 + saves•0.3 + views•0.1`, the trending score itself. Trending is currently ranking domains on a fraction of real engagement, biased toward whichever surface happens to be instrumented.

`domain-item/page.tsx:57,76,78,85` • zero `setAnalytics` in the other four

DRIFT The copies have already diverged

They are not identical any more. The seller portfolio calls `setSelectedDomain(item)` in both branches and uses `setShowModal(true)` directly; the slider and card call `handleShow()`. The card passes a `domainId` into the wallet modal, the slider passes none. Two use `ROUTER`, one uses `router`.

That drift is the mechanism by which the analytics gap appeared, and it will keep producing gaps. Item 8 has to touch all five of these, which is the practical reason to do 21 first.

`slider.tsx:157-183` • `seller-portfolio/[slug].tsx:272-303` • `watchlist/page.tsx:147-178`

PERFORMANCE Every card mounts its own modal pair

All five surfaces render a `<BuyNowModal>` and a `<WalletConnect>` per item, inside the map. A twelve-card grid mounts twenty-four modals. `WalletConnect` calls `createAppKit(...)` in its component body, so that re-runs on every render of every card.

`domain-item/page.tsx:176-196` • `watchlist/page.tsx:181-186`

What to replace it with

Two pieces, both following patterns already in this codebase — the `provider-manager` singleton and the existing `useOnScreen` hook.

1. A `useBuyIntent()` hook that owns the whole ladder: read the token, read the wallet address, check the provider, fire the analytics event, and return either `{action:'login'}`, `{action:'add-wallet'}`, `{action:'connect'}` or `{action:'buy'}`. Every surface calls `onClick={() => buy(domain)}` and nothing else. Analytics becomes impossible to forget because it lives inside the hook, not at the call site.
2. One modal host at the app root, driven by that hook's state, instead of a pair per card. Twenty-four mounted modals become two, and `createAppKit` runs once for the app rather than once per card render.

```
// one hook, five call sites, analytics guaranteed
const { buy } = useBuyIntent();

// in every surface, identically:
<button onClick={() => buy(domain)}>Buy Now</button>
```

Sequence these together, 21 before 8. Item 8 rewrites the handler that all five gates lead into; doing it while five copies still exist means either five edits or a fix that only lands on one surface. Consolidating first turns item 8 into a single-file change.

Doing 21 first also backfills the analytics hole immediately — which matters because item 14 rebuilds seller analytics on top of this data. Fixing the dashboard while four of five surfaces still record nothing would just render incomplete numbers more attractively.

→ Suggested order

1. **Consolidate the gate (21) — ~4 days.** Extract `useBuyIntent()`, move the modals to a single host, delete the five copies. Analytics coverage goes from one surface to five on the same commit.
2. **Rewrite the handler (8) — ~1 week.** Now a single file. Reorder to confirm-then-persist-then-claim, check `receipt.status`, disable the button by phase, switch chains inline, fix the balance comparison, and classify errors properly.
3. **Then verify against the reconciler.** The backend already validates the `NewSale` event including price within a one-cent epsilon. Once the frontend stops reporting unconfirmed purchases, that cron becomes a true safety net rather than the thing quietly cleaning up after optimistic UI.

Worth checking during the work: whether the 2% buffer overage is refunded on-chain. If the contract keeps the difference rather than returning it, buyers are systematically overpaying by up to 2% and that is a pricing issue rather than a UI one.